

Treasure Hunt Game Notebook

Read and Review Your Starter Code

The theme of this project is a popular treasure hunt game in which the player needs to find the treasure before the pirate does. While you will not be developing the entire game, you will write the part of the game that represents the intelligent agent, which is a pirate in this case. The pirate will try to find the optimal path to the treasure using deep Q-learning.

To Begin: Use this **TreasureHuntGame_starterCode.ipynb** file to complete your assignment.

You have been provided with two Python classes and this notebook to help you with this assignment. The first class, **TreasureMaze.py**, represents the environment, which includes a maze object defined as a matrix. The second class, **GameExperience.py**, stores the episodes – that is, all the states that come in between the initial state and the terminal state. This is later used by the agent for learning by experience, called "exploration". This notebook shows how to play a game. Your task is to complete the deep Q-learning implementation in the `qtrain()` function for which a skeleton implementation has been provided.

NOTE: The code block you will need to complete will have **#TODO** as a header. First, read and review the next few code and instruction blocks to understand the code that you have been given.

Installations The following command will install the necessary Python libraries to necessary to run this application. If you see a "[notice] A new release of pip is available: 23.1.2 -> 25.2" at the end of the installation, you may disregard that statement.

In [29]: `!pip install -r requirements.txt`

```

Requirement already satisfied: numpy==1.26.4 in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from -r requirements.txt (line 1)) (1.26.4)
Requirement already satisfied: keras in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from -r requirements.txt (line 2)) (3.4.1)
Requirement already satisfied: absl-py in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from keras->-r requirements.txt (line 2)) (2.1.0)
Requirement already satisfied: rich in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from keras->-r requirements.txt (line 2)) (13.7.1)
Requirement already satisfied: namex in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from keras->-r requirements.txt (line 2)) (0.0.8)
Requirement already satisfied: h5py in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from keras->-r requirements.txt (line 2)) (3.11.0)
Requirement already satisfied: optree in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from keras->-r requirements.txt (line 2)) (0.12.1)
Requirement already satisfied: ml-dtypes in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from keras->-r requirements.txt (line 2)) (0.4.0)
Requirement already satisfied: packaging in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from keras->-r requirements.txt (line 2)) (24.1)
Requirement already satisfied: typing-extensions>=4.5.0 in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from optree->keras->-r requirements.txt (line 2)) (4.12.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from rich->keras->-r requirements.txt (line 2)) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from rich->keras->-r requirements.txt (line 2)) (2.18.0)
Requirement already satisfied: mdurl~=0.1 in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from markdown-it-py>=2.2.0->rich->keras->-r requirements.txt (line 2)) (0.1.2)

```

[notice] A new release of pip is available: 24.1.2 -> 26.1.2

[notice] To update, run: `pip install --upgrade pip`

Tensorflow CPU Acceleration Warning

GPU/CUDA/Memory Warnings/Errors: You may receive some errors referencing that GPUs will not be used, CUDA could not be found, or free system memory allocation errors. These and a few others, are standard errors that can be ignored here as they are environment based.

Example messages:

- oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders
- WARNING: All log messages before `absl::InitializeLog()` is called are written to STDERR

```
In [30]: from __future__ import print_function
import os, sys, time, datetime, json, random
```

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import clone_model
from keras.models import Sequential
from keras.layers import Dense, Activation, PReLU
from keras.optimizers import SGD, Adam, RMSprop
import matplotlib.pyplot as plt
from TreasureMaze import TreasureMaze
from GameExperience import GameExperience
%matplotlib inline
```

Maze Object Generation

NOTE: The following code block contains an 8x8 matrix that will be used as a maze object:

```
In [31]: maze = np.array([
    [ 1.,  0.,  1.,  1.,  1.,  1.,  1.,  1.],
    [ 1.,  0.,  1.,  1.,  1.,  0.,  1.,  1.],
    [ 1.,  1.,  1.,  1.,  0.,  1.,  0.,  1.],
    [ 1.,  1.,  1.,  0.,  1.,  1.,  1.,  1.],
    [ 1.,  1.,  0.,  1.,  1.,  1.,  1.,  1.],
    [ 1.,  1.,  1.,  0.,  1.,  0.,  0.,  0.],
    [ 1.,  1.,  1.,  0.,  1.,  1.,  1.,  1.],
    [ 1.,  1.,  1.,  1.,  0.,  1.,  1.,  1.]
])
```

Helper Functions and Global Variables

This **show()** helper function allows a visual representation of the maze object:

```
In [32]: def show(qmaze):
    plt.grid('on')
    nrows, ncols = qmaze.maze.shape
    ax = plt.gca()
    ax.set_xticks(np.arange(0.5, nrows, 1))
    ax.set_yticks(np.arange(0.5, ncols, 1))
    ax.set_xticklabels([])
    ax.set_yticklabels([])
    canvas = np.copy(qmaze.maze)
    for row, col in qmaze.visited:
        canvas[row, col] = 0.6
    pirate_row, pirate_col, _ = qmaze.state
    canvas[pirate_row, pirate_col] = 0.3 # pirate cell
    canvas[nrows-1, ncols-1] = 0.9 # treasure cell
    img = plt.imshow(canvas, interpolation='none', cmap='gray')
    return img
```

The **pirate agent** can move in four directions: left, right, up, and down.

Note: While the agent primarily learns by experience through exploitation, often, the agent can choose to explore the environment to find previously undiscovered paths. This is called "exploration" and is defined by epsilon. This value is the **EXPLORATION** values from the Cartpole assignment. The hyperparameters are provided here and used in the **qtrain()** method. You are encouraged to try various values for the exploration factor and see how the algorithm performs.

```
In [33]: LEFT = 0
UP = 1
RIGHT = 2
DOWN = 3

# Exploration factor
epsilon = 1.0
epsilon_min = 0.05
epsilon_decay = 0.995
patience = 10

# Actions dictionary
actions_dict = {
    LEFT: 'left',
    UP: 'up',
    RIGHT: 'right',
    DOWN: 'down',
}

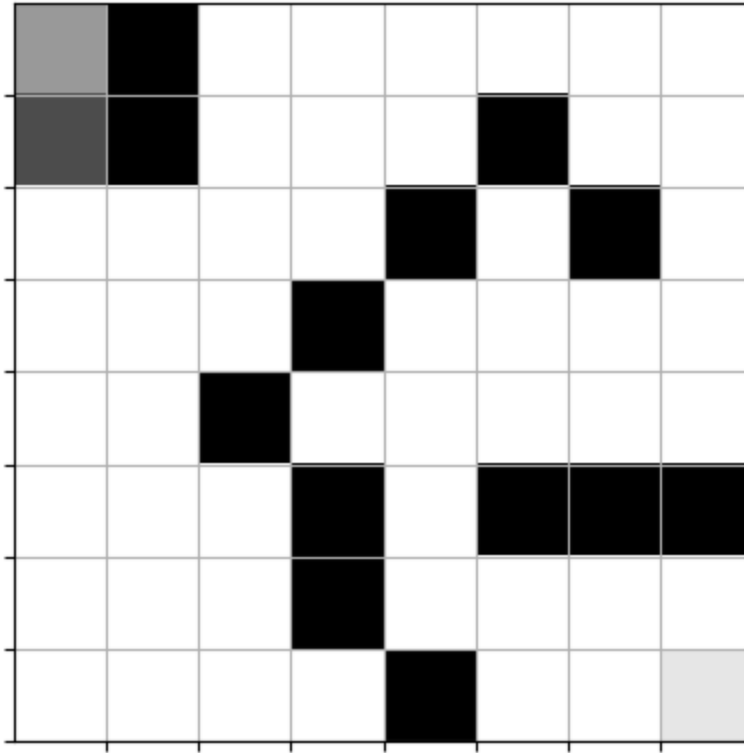
num_actions = len(actions_dict)
```

The sample code block and output below show creating a maze object and performing one action (DOWN), which returns the reward. The resulting updated environment is visualized.

```
In [34]: qmaze = TreasureMaze(maze)
canvas, reward, game_over = qmaze.act(DOWN)
print("reward=", reward)
show(qmaze)
```

reward= -0.04

Out[34]: <matplotlib.image.AxesImage at 0x73a754164790>



NOTE: This `play_game()` function simulates a full game based on the provided trained model. The other parameters include the TreasureMaze object, the starting position of the pirate and max amount of steps to make sure the code does not get stuck in a loop.

```
In [35]: def play_game(model, qmaze, pirate_cell, max_steps=None):
    qmaze.reset(pirate_cell)
    envstate = qmaze.observe()
    steps = 0
    if max_steps is None:
        max_steps = qmaze.maze.size * 4 # safety cutoff

    while steps < max_steps:
        state = np.asarray(envstate, dtype=np.float32)
        if state.ndim == 1:
            state = np.expand_dims(state, axis=0)

        q_values = model(state, training=False).numpy()
        action = np.argmax(q_values[0])

        envstate, reward, game_status = qmaze.act(action)
        steps += 1

        if game_status == 'win':
            return True
        elif game_status == 'lose':
            return False

    return False # timed out with no result
```

Note: This `completion_check()` function helps you to determine whether the pirate can win any game at all. If your maze is not well designed, the pirate may not win any game at all. In this case, your training would not yield any result. The provided maze in this notebook ensures that there is a path to win and you can run this method to check.

```
In [36]: def completion_check(model, maze_or_qmaze, max_steps=None):
# Accept either raw numpy maze or TreasureMaze instance
if isinstance(maze_or_qmaze, TreasureMaze):
    qmaze = maze_or_qmaze
else:
    qmaze = TreasureMaze(maze_or_qmaze)

for cell in qmaze.free_cells:
    if not qmaze.valid_actions(cell):
        continue
    if not play_game(model, qmaze, cell, max_steps=max_steps):
        return False
return True
```

Note: The `build_model()` function in the block below will build the neural network model. Review the code and note the number of layers, as well as the activation, optimizer, and loss functions that are used to train the model.

```
In [37]: def build_model(maze):
model = Sequential()
model.add(Dense(maze.size, input_shape=(maze.size,)))
model.add(PReLU())
model.add(Dense(maze.size))
model.add(PReLU())
model.add(Dense(num_actions))
model.compile(optimizer='adam', loss='mse')
return model
```

Note: This `train_step()` helper function in the block below is used to help predict Q-values (quality values) in the current model to see how good each action is in a given state and improve the Q-network by reducing the gap between what is predicted and what should have been predicted.

If you're interested in reading up on the `@tf.function`, which is a decorator for Tensorflow to run this code into a TensorFlow computation graph, please refer to this link: https://www.tensorflow.org/guide/intro_to_graphs

Tensorflow GPU Warning

You will see a **warning in red** "INTERNAL: CUDA Runtime error: Failed call to cudaGetRuntimeVersion: Error loading CUDA libraries. GPU will not be used.". This is simply coming from **Tensorflow skipping using GPU for this assignment**.

```
In [38]: loss_fn = tf.keras.losses.MeanSquaredError()
optimizer = tf.keras.optimizers.Adam()

@tf.function
def train_step(x, y):
    with tf.GradientTape() as tape:
        q_values = model(x, training=True)
        loss = loss_fn(y, q_values)
    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables))
    return loss
```

#TODO: Complete the Q-Training Algorithm Code Block

This is your deep Q-learning implementation. The goal of your deep Q-learning implementation is to find the best possible navigation sequence that results in reaching the treasure cell while maximizing the reward. In your implementation, you need to determine the optimal number of epochs to achieve a 100% win rate.

Pseudocode:

For each epoch: Reset the environment at a random starting cell agent_cell = randomly select a free cell

Hint: Review the reset method in the TreasureMaze.py class.

Set the initial environment state
env_state should reference the environment's current state
Hint: Review the observe method in the TreasureMaze.py class.

While game status is not game over:
 previous_envstate = env_state
 Decide on an action:
 - If possible, take a random valid exploration action
and
 randomly choose action (left, right, up, down)
 and assign it to an action variable
 - Else, pick the best exploitation action from the
model and assign it to an action variable
 Hint: Review the predict method in the
GameExperience.py class.

Retrieve the values below from the `act()` method.
`env_state, reward, game_status = qmaze.act(action)`
 Hint: Review the `act` method in the `TreasureMaze.py` class.

Track the wins and losses from the `game_status` using
`win_history`

Store the episode below in the Experience replay object
`episode = [previous_envstate, action, reward, envstate, game_status]`
 Hint: Review the `remember` method in the `GameExperience.py` class.

Train neural network model and evaluate loss
 Hint: Call `GameExperience.get_data` to retrieve training data (input and target)
 and pass to the `train_step` method and assign it to `batch_loss` and append to the loss variable

If the win rate is above the threshold and your model passes the completion check, that would be your epoch.

Note: A 100% win rate **DOES NOT EXPLICITLY MEAN** that you have solved the maze. It simply indicates that during the last evaluation, the pirate *happened* to get to the treasure. Be sure to utilise the **`completion_check()`** function to validate your pirate found the treasure at every starting point and consistently!

You will need to complete the section starting with `#START_HERE`. Please use the pseudocode above as guidance.

```
In [39]: def qtrain(model, maze, **opt):
# exploration factor
global epsilon

# Number of epochs
n_epoch = opt.get('n_epoch', 15000)

# Maximum meory to store episodes
max_memory = opt.get('max_memory', 1000)

# Maximum data size for training
data_size = opt.get('data_size', 50)

# Frequency of target network updates
target_update_freq = opt.get('target_update_freq', 50)

# Start time
start_time = datetime.datetime.now()

# Construct environment/game from numpy array: maze (see argument above)
qmaze = TreasureMaze(maze)
```



```

# Target Network to better guide training
target_model = clone_model(model)
target_model.set_weights(model.get_weights())

# Initialize experience replay object
experience = GameExperience(model, target_model, max_memory=max_memory)

win_history = [] # history of win/lose game
hsize = qmaze.maze.size // 2 #history window size
win_rate = 0.0

# =====START_HERE=====
loss = 0
n_episodes = 0;
for epoch in range(n_epoch):
    # Win rate over last hsize steps
    win_rate = sum(win_history[-hsize:]) / hsize if len(win_history) >= hsize else 0

    # Print the epoch, loss, episode, win count, win rate, and time for each episode
    dt = datetime.datetime.now() - start_time
    t = format_time(dt.total_seconds())
    print("Epoch: {:03d}/{:d} | Loss: {:.4f} | Episodes: {:d} | Win count: {:d} | Win rate: {:.2f} | Time: {}".format(
        epoch, n_epoch-1, loss, n_episodes, sum(win_history), win_rate, t))
    #creating the variables
    agent_cell = np.random.randint(0, 7, 2)
    qmaze.reset([0,0])
    Env_state = qmaze.observe()

    #creating while loop for the function while it's going on
    while( qmaze.game_status() == "not_over"):
        #creation of variables
        Previous_state = Env_state
        Valid_action= qmaze.valid_actions()
        # preforms it moves and decides if it goes in a random direction or not.
        if np.random.rand() < epsilon:
            action = random.choice(Valid_action)
        else:
            action = np.argmax(experience.predict(Env_state))

        #preforms actions
        Env_state, reward, game_status = qmaze.act(action)
        #increases episode count due to it costing an episode
        n_episodes+=1

        #remembering the episode
        episode = [Previous_state, action, reward, Env_state , game_status]
        experience.remember(episode)
        inputs,targets = experience.get_data()

        history = model.fit(inputs,targets,epochs = 8, batch_size =24, verbose=0)
        loss = history.history['loss'][-1]
        if episode[4] == "win":
            win_history.append(1)
            win_rate = sum(win_history) / len(win_history)
            break

```

```

        elif episode[4] == "lose":
            win_history.append(0)
            win_rate = sum(win_history) / len(win_history)
            break

    # Check if training has exhausted all free cells and if in all
    # cases the agent won
    if win_rate > 0.9:
        epsilon = 0.05
    else:
        epsilon = max(epsilon * epsilon_decay, epsilon_min)

    if win_rate >= 0.999 and completion_check(model, maze):
        print(f"Reached 100% win rate at epoch {epoch}")
        break

total_time = format_time((datetime.datetime.now() - start_time).total_seconds())
print("Training complete in:", total_time)

# This is a small utility for printing readable time strings:
def format_time(seconds):
    if seconds < 400:
        s = float(seconds)
        return "%.1f seconds" % (s,)
    elif seconds < 4000:
        m = seconds / 60.0
        return "%.2f minutes" % (m,)
    else:
        h = seconds / 3600.0
        return "%.2f hours" % (h,)

```

Test Your Model

Now we will start testing the deep Q-learning implementation. To begin, select **Cell**, then **Run All** from the menu bar. This will run your notebook. As it runs, you should see output begin to appear beneath the next few cells. The code below creates an **instance** of TreasureMaze. This does not show your actual training done.

```

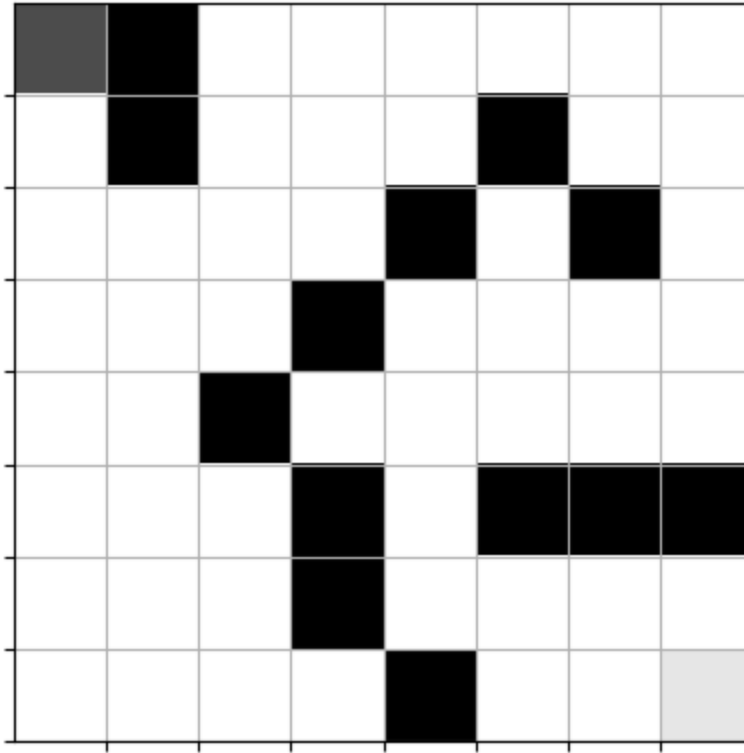
In [40]: qmaze = TreasureMaze(maze)
         show(qmaze)

```

```

Out[40]: <matplotlib.image.AxesImage at 0x73a75bf74b10>

```



In the next code block, you will build your model using the **build_model** function and train it using deep Q-learning. Note: This step takes several minutes to fully run.

WARNING If you did not attempt the assignment, the code **will** error out at this section.

```
In [ ]: model = build_model(maze)
        qtrain(model, maze, n_epoch=1000, max_memory=8*maze.size, data_size=32, target_upd

/home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Epoch: 000/999 | Loss: 0.0000 | Episodes: 0 | Win count: 0 | Win rate: 0.000 | time: 0.0 seconds
Epoch: 001/999 | Loss: 0.0018 | Episodes: 161 | Win count: 0 | Win rate: 0.000 | time: 42.7 seconds
Epoch: 002/999 | Loss: 0.0012 | Episodes: 305 | Win count: 0 | Win rate: 0.000 | time: 79.5 seconds
Epoch: 003/999 | Loss: 0.0016 | Episodes: 455 | Win count: 0 | Win rate: 0.000 | time: 118.2 seconds
Epoch: 004/999 | Loss: 0.0014 | Episodes: 598 | Win count: 0 | Win rate: 0.000 | time: 155.0 seconds
Epoch: 005/999 | Loss: 0.0014 | Episodes: 748 | Win count: 0 | Win rate: 0.000 | time: 194.1 seconds
Epoch: 006/999 | Loss: 0.0009 | Episodes: 898 | Win count: 0 | Win rate: 0.000 | time: 233.1 seconds
Epoch: 007/999 | Loss: 0.0012 | Episodes: 1047 | Win count: 0 | Win rate: 0.000 | time: 271.9 seconds
Epoch: 008/999 | Loss: 0.0016 | Episodes: 1196 | Win count: 0 | Win rate: 0.000 | time: 310.9 seconds
Epoch: 009/999 | Loss: 0.0005 | Episodes: 1348 | Win count: 0 | Win rate: 0.000 | time: 350.6 seconds
Epoch: 010/999 | Loss: 0.0014 | Episodes: 1492 | Win count: 0 | Win rate: 0.000 | time: 388.1 seconds
Epoch: 011/999 | Loss: 0.0016 | Episodes: 1638 | Win count: 0 | Win rate: 0.000 | time: 7.10 minutes
Epoch: 012/999 | Loss: 0.0011 | Episodes: 1788 | Win count: 0 | Win rate: 0.000 | time: 7.75 minutes
Epoch: 013/999 | Loss: 0.0008 | Episodes: 1940 | Win count: 0 | Win rate: 0.000 | time: 8.38 minutes
Epoch: 014/999 | Loss: 0.0016 | Episodes: 2093 | Win count: 0 | Win rate: 0.000 | time: 9.00 minutes
Epoch: 015/999 | Loss: 0.0018 | Episodes: 2240 | Win count: 0 | Win rate: 0.000 | time: 9.59 minutes
Epoch: 016/999 | Loss: 0.0016 | Episodes: 2398 | Win count: 0 | Win rate: 0.000 | time: 10.23 minutes
Epoch: 017/999 | Loss: 0.0024 | Episodes: 2552 | Win count: 0 | Win rate: 0.000 | time: 10.85 minutes
Epoch: 018/999 | Loss: 0.0011 | Episodes: 2699 | Win count: 0 | Win rate: 0.000 | time: 11.44 minutes
Epoch: 019/999 | Loss: 0.0013 | Episodes: 2844 | Win count: 0 | Win rate: 0.000 | time: 12.04 minutes
Epoch: 020/999 | Loss: 0.0010 | Episodes: 2995 | Win count: 0 | Win rate: 0.000 | time: 12.65 minutes
Epoch: 021/999 | Loss: 0.0018 | Episodes: 3150 | Win count: 0 | Win rate: 0.000 | time: 13.29 minutes
Epoch: 022/999 | Loss: 0.0014 | Episodes: 3302 | Win count: 0 | Win rate: 0.000 | time: 13.90 minutes
Epoch: 023/999 | Loss: 0.0019 | Episodes: 3454 | Win count: 0 | Win rate: 0.000 | time: 14.52 minutes
Epoch: 024/999 | Loss: 0.0014 | Episodes: 3605 | Win count: 0 | Win rate: 0.000 | time: 15.14 minutes
Epoch: 025/999 | Loss: 0.0007 | Episodes: 3755 | Win count: 0 | Win rate: 0.000 | time: 15.76 minutes
Epoch: 026/999 | Loss: 0.0012 | Episodes: 3899 | Win count: 0 | Win rate: 0.000 | time: 16.34 minutes
Epoch: 027/999 | Loss: 0.0012 | Episodes: 4045 | Win count: 0 | Win rate: 0.000 | time: 16.94 minutes

Epoch: 028/999 | Loss: 0.0009 | Episodes: 4195 | Win count: 0 | Win rate: 0.000 | time: 17.56 minutes
Epoch: 029/999 | Loss: 0.0017 | Episodes: 4345 | Win count: 0 | Win rate: 0.000 | time: 18.17 minutes
Epoch: 030/999 | Loss: 0.0018 | Episodes: 4490 | Win count: 1 | Win rate: 0.000 | time: 18.75 minutes
Epoch: 031/999 | Loss: 0.0014 | Episodes: 4630 | Win count: 2 | Win rate: 0.000 | time: 19.34 minutes
Epoch: 032/999 | Loss: 0.0013 | Episodes: 4775 | Win count: 2 | Win rate: 0.062 | time: 19.94 minutes
Epoch: 033/999 | Loss: 0.0012 | Episodes: 4925 | Win count: 2 | Win rate: 0.062 | time: 20.56 minutes
Epoch: 034/999 | Loss: 0.0015 | Episodes: 5080 | Win count: 2 | Win rate: 0.062 | time: 21.20 minutes
Epoch: 035/999 | Loss: 0.0012 | Episodes: 5239 | Win count: 2 | Win rate: 0.062 | time: 21.85 minutes
Epoch: 036/999 | Loss: 0.0007 | Episodes: 5388 | Win count: 2 | Win rate: 0.062 | time: 22.46 minutes
Epoch: 037/999 | Loss: 0.0011 | Episodes: 5524 | Win count: 3 | Win rate: 0.094 | time: 23.02 minutes
Epoch: 038/999 | Loss: 0.0026 | Episodes: 5671 | Win count: 3 | Win rate: 0.094 | time: 23.62 minutes
Epoch: 039/999 | Loss: 0.0010 | Episodes: 5815 | Win count: 4 | Win rate: 0.125 | time: 24.20 minutes
Epoch: 040/999 | Loss: 0.0018 | Episodes: 5966 | Win count: 4 | Win rate: 0.125 | time: 24.82 minutes
Epoch: 041/999 | Loss: 0.0017 | Episodes: 6115 | Win count: 4 | Win rate: 0.125 | time: 25.43 minutes
Epoch: 042/999 | Loss: 0.0010 | Episodes: 6267 | Win count: 4 | Win rate: 0.125 | time: 26.05 minutes
Epoch: 043/999 | Loss: 0.0011 | Episodes: 6422 | Win count: 4 | Win rate: 0.125 | time: 26.70 minutes
Epoch: 044/999 | Loss: 0.0013 | Episodes: 6576 | Win count: 4 | Win rate: 0.125 | time: 27.35 minutes
Epoch: 045/999 | Loss: 0.0022 | Episodes: 6721 | Win count: 4 | Win rate: 0.125 | time: 27.95 minutes
Epoch: 046/999 | Loss: 0.0009 | Episodes: 6865 | Win count: 4 | Win rate: 0.125 | time: 28.54 minutes
Epoch: 047/999 | Loss: 0.0011 | Episodes: 7002 | Win count: 5 | Win rate: 0.156 | time: 29.10 minutes
Epoch: 048/999 | Loss: 0.0017 | Episodes: 7160 | Win count: 5 | Win rate: 0.156 | time: 29.75 minutes
Epoch: 049/999 | Loss: 0.0009 | Episodes: 7310 | Win count: 5 | Win rate: 0.156 | time: 30.36 minutes
Epoch: 050/999 | Loss: 0.0008 | Episodes: 7463 | Win count: 5 | Win rate: 0.156 | time: 30.99 minutes
Epoch: 051/999 | Loss: 0.0008 | Episodes: 7611 | Win count: 5 | Win rate: 0.156 | time: 31.60 minutes
Epoch: 052/999 | Loss: 0.0010 | Episodes: 7703 | Win count: 6 | Win rate: 0.188 | time: 31.97 minutes
Epoch: 053/999 | Loss: 0.0015 | Episodes: 7845 | Win count: 6 | Win rate: 0.188 | time: 32.55 minutes
Epoch: 054/999 | Loss: 0.0009 | Episodes: 7995 | Win count: 7 | Win rate: 0.219 | time: 33.15 minutes
Epoch: 055/999 | Loss: 0.0020 | Episodes: 8139 | Win count: 7 | Win rate: 0.219 | time: 33.75 minutes

Epoch: 056/999 | Loss: 0.0015 | Episodes: 8290 | Win count: 7 | Win rate: 0.219 | time: 34.39 minutes
Epoch: 057/999 | Loss: 0.0017 | Episodes: 8433 | Win count: 7 | Win rate: 0.219 | time: 34.99 minutes
Epoch: 058/999 | Loss: 0.0009 | Episodes: 8574 | Win count: 7 | Win rate: 0.219 | time: 35.58 minutes
Epoch: 059/999 | Loss: 0.0013 | Episodes: 8733 | Win count: 8 | Win rate: 0.250 | time: 36.23 minutes
Epoch: 060/999 | Loss: 0.0014 | Episodes: 8879 | Win count: 9 | Win rate: 0.281 | time: 36.83 minutes
Epoch: 061/999 | Loss: 0.0013 | Episodes: 9039 | Win count: 9 | Win rate: 0.281 | time: 37.49 minutes
Epoch: 062/999 | Loss: 0.0017 | Episodes: 9195 | Win count: 10 | Win rate: 0.281 | time: 38.13 minutes
Epoch: 063/999 | Loss: 0.0019 | Episodes: 9335 | Win count: 11 | Win rate: 0.281 | time: 38.71 minutes
Epoch: 064/999 | Loss: 0.0018 | Episodes: 9479 | Win count: 11 | Win rate: 0.281 | time: 39.29 minutes
Epoch: 065/999 | Loss: 0.0017 | Episodes: 9633 | Win count: 12 | Win rate: 0.312 | time: 39.92 minutes
Epoch: 066/999 | Loss: 0.0015 | Episodes: 9783 | Win count: 12 | Win rate: 0.312 | time: 40.54 minutes
Epoch: 067/999 | Loss: 0.0012 | Episodes: 9926 | Win count: 12 | Win rate: 0.312 | time: 41.14 minutes
Epoch: 068/999 | Loss: 0.0010 | Episodes: 10076 | Win count: 12 | Win rate: 0.312 | time: 41.81 minutes
Epoch: 069/999 | Loss: 0.0012 | Episodes: 10220 | Win count: 12 | Win rate: 0.281 | time: 42.44 minutes
Epoch: 070/999 | Loss: 0.0016 | Episodes: 10362 | Win count: 12 | Win rate: 0.281 | time: 43.05 minutes
Epoch: 071/999 | Loss: 0.0010 | Episodes: 10509 | Win count: 12 | Win rate: 0.250 | time: 43.68 minutes
Epoch: 072/999 | Loss: 0.0014 | Episodes: 10652 | Win count: 12 | Win rate: 0.250 | time: 44.27 minutes
Epoch: 073/999 | Loss: 0.0010 | Episodes: 10781 | Win count: 13 | Win rate: 0.281 | time: 44.80 minutes
Epoch: 074/999 | Loss: 0.0003 | Episodes: 10923 | Win count: 13 | Win rate: 0.281 | time: 45.39 minutes
Epoch: 075/999 | Loss: 0.0007 | Episodes: 11084 | Win count: 13 | Win rate: 0.281 | time: 46.06 minutes
Epoch: 076/999 | Loss: 0.0017 | Episodes: 11183 | Win count: 14 | Win rate: 0.312 | time: 46.47 minutes
Epoch: 077/999 | Loss: 0.0022 | Episodes: 11319 | Win count: 15 | Win rate: 0.344 | time: 47.02 minutes
Epoch: 078/999 | Loss: 0.0011 | Episodes: 11461 | Win count: 15 | Win rate: 0.344 | time: 47.61 minutes
Epoch: 079/999 | Loss: 0.0015 | Episodes: 11606 | Win count: 15 | Win rate: 0.312 | time: 48.20 minutes
Epoch: 080/999 | Loss: 0.0008 | Episodes: 11759 | Win count: 15 | Win rate: 0.312 | time: 48.85 minutes
Epoch: 081/999 | Loss: 0.0011 | Episodes: 11830 | Win count: 16 | Win rate: 0.344 | time: 49.18 minutes
Epoch: 082/999 | Loss: 0.0019 | Episodes: 11929 | Win count: 17 | Win rate: 0.375 | time: 49.63 minutes
Epoch: 083/999 | Loss: 0.0014 | Episodes: 12025 | Win count: 18 | Win rate: 0.406 | time: 50.04 minutes

Epoch: 084/999 | Loss: 0.0012 | Episodes: 12128 | Win count: 19 | Win rate: 0.406 | time: 50.48 minutes
Epoch: 085/999 | Loss: 0.0008 | Episodes: 12270 | Win count: 19 | Win rate: 0.406 | time: 51.09 minutes
Epoch: 086/999 | Loss: 0.0014 | Episodes: 12390 | Win count: 20 | Win rate: 0.406 | time: 51.59 minutes
Epoch: 087/999 | Loss: 0.0010 | Episodes: 12455 | Win count: 21 | Win rate: 0.438 | time: 51.86 minutes
Epoch: 088/999 | Loss: 0.0012 | Episodes: 12599 | Win count: 22 | Win rate: 0.469 | time: 52.45 minutes
Epoch: 089/999 | Loss: 0.0014 | Episodes: 12651 | Win count: 23 | Win rate: 0.500 | time: 52.67 minutes
Epoch: 090/999 | Loss: 0.0011 | Episodes: 12712 | Win count: 24 | Win rate: 0.531 | time: 52.92 minutes
Epoch: 091/999 | Loss: 0.0014 | Episodes: 12780 | Win count: 25 | Win rate: 0.531 | time: 53.20 minutes
Epoch: 092/999 | Loss: 0.0013 | Episodes: 12943 | Win count: 26 | Win rate: 0.531 | time: 53.87 minutes
Epoch: 093/999 | Loss: 0.0013 | Episodes: 13004 | Win count: 27 | Win rate: 0.562 | time: 54.12 minutes
Epoch: 094/999 | Loss: 0.0015 | Episodes: 13154 | Win count: 27 | Win rate: 0.531 | time: 54.74 minutes
Epoch: 095/999 | Loss: 0.0011 | Episodes: 13198 | Win count: 28 | Win rate: 0.531 | time: 54.92 minutes
Epoch: 096/999 | Loss: 0.0014 | Episodes: 13360 | Win count: 29 | Win rate: 0.562 | time: 55.58 minutes
Epoch: 097/999 | Loss: 0.0009 | Episodes: 13502 | Win count: 29 | Win rate: 0.531 | time: 56.18 minutes
Epoch: 098/999 | Loss: 0.0013 | Episodes: 13646 | Win count: 29 | Win rate: 0.531 | time: 56.80 minutes
Epoch: 099/999 | Loss: 0.0001 | Episodes: 13786 | Win count: 29 | Win rate: 0.531 | time: 57.40 minutes
Epoch: 100/999 | Loss: 0.0005 | Episodes: 13886 | Win count: 30 | Win rate: 0.562 | time: 57.82 minutes
Epoch: 101/999 | Loss: 0.0010 | Episodes: 13977 | Win count: 31 | Win rate: 0.594 | time: 58.21 minutes
Epoch: 102/999 | Loss: 0.0014 | Episodes: 14022 | Win count: 32 | Win rate: 0.625 | time: 58.39 minutes
Epoch: 103/999 | Loss: 0.0010 | Episodes: 14098 | Win count: 33 | Win rate: 0.656 | time: 58.71 minutes
Epoch: 104/999 | Loss: 0.0019 | Episodes: 14202 | Win count: 34 | Win rate: 0.688 | time: 59.16 minutes
Epoch: 105/999 | Loss: 0.0012 | Episodes: 14346 | Win count: 34 | Win rate: 0.656 | time: 59.75 minutes
Epoch: 106/999 | Loss: 0.0016 | Episodes: 14443 | Win count: 35 | Win rate: 0.688 | time: 60.15 minutes
Epoch: 107/999 | Loss: 0.0019 | Episodes: 14487 | Win count: 36 | Win rate: 0.719 | time: 60.34 minutes
Epoch: 108/999 | Loss: 0.0012 | Episodes: 14628 | Win count: 36 | Win rate: 0.688 | time: 60.92 minutes
Epoch: 109/999 | Loss: 0.0013 | Episodes: 14773 | Win count: 36 | Win rate: 0.656 | time: 61.52 minutes
Epoch: 110/999 | Loss: 0.0009 | Episodes: 14850 | Win count: 37 | Win rate: 0.688 | time: 61.83 minutes
Epoch: 111/999 | Loss: 0.0008 | Episodes: 14992 | Win count: 37 | Win rate: 0.688 | time: 62.42 minutes

Epoch: 112/999 | Loss: 0.0008 | Episodes: 15127 | Win count: 38 | Win rate: 0.719 |
time: 62.97 minutes
Epoch: 113/999 | Loss: 0.0013 | Episodes: 15269 | Win count: 38 | Win rate: 0.688 |
time: 63.57 minutes
Epoch: 114/999 | Loss: 0.0016 | Episodes: 15362 | Win count: 39 | Win rate: 0.688 |
time: 64.00 minutes
Epoch: 115/999 | Loss: 0.0014 | Episodes: 15435 | Win count: 40 | Win rate: 0.688 |
time: 64.32 minutes
Epoch: 116/999 | Loss: 0.0017 | Episodes: 15558 | Win count: 41 | Win rate: 0.688 |
time: 64.87 minutes
Epoch: 117/999 | Loss: 0.0018 | Episodes: 15710 | Win count: 42 | Win rate: 0.719 |
time: 65.54 minutes
Epoch: 118/999 | Loss: 0.0017 | Episodes: 15807 | Win count: 43 | Win rate: 0.719 |
time: 65.96 minutes
Epoch: 119/999 | Loss: 0.0015 | Episodes: 15946 | Win count: 44 | Win rate: 0.719 |
time: 66.57 minutes
Epoch: 120/999 | Loss: 0.0011 | Episodes: 16087 | Win count: 44 | Win rate: 0.688 |
time: 1.12 hours
Epoch: 121/999 | Loss: 0.0012 | Episodes: 16216 | Win count: 45 | Win rate: 0.688 |
time: 1.13 hours
Epoch: 122/999 | Loss: 0.0010 | Episodes: 16359 | Win count: 45 | Win rate: 0.656 |
time: 1.14 hours
Epoch: 123/999 | Loss: 0.0022 | Episodes: 16514 | Win count: 45 | Win rate: 0.625 |
time: 1.15 hours
Epoch: 124/999 | Loss: 0.0014 | Episodes: 16585 | Win count: 46 | Win rate: 0.625 |
time: 1.15 hours
Epoch: 125/999 | Loss: 0.0007 | Episodes: 16641 | Win count: 47 | Win rate: 0.625 |
time: 1.16 hours
Epoch: 126/999 | Loss: 0.0007 | Episodes: 16783 | Win count: 47 | Win rate: 0.625 |
time: 1.17 hours
Epoch: 127/999 | Loss: 0.0011 | Episodes: 16872 | Win count: 48 | Win rate: 0.625 |
time: 1.17 hours
Epoch: 128/999 | Loss: 0.0017 | Episodes: 16929 | Win count: 49 | Win rate: 0.625 |
time: 1.18 hours
Epoch: 129/999 | Loss: 0.0016 | Episodes: 17019 | Win count: 50 | Win rate: 0.656 |
time: 1.18 hours
Epoch: 130/999 | Loss: 0.0017 | Episodes: 17098 | Win count: 51 | Win rate: 0.688 |
time: 1.19 hours
Epoch: 131/999 | Loss: 0.0013 | Episodes: 17164 | Win count: 52 | Win rate: 0.719 |
time: 1.19 hours
Epoch: 132/999 | Loss: 0.0015 | Episodes: 17214 | Win count: 53 | Win rate: 0.719 |
time: 1.20 hours
Epoch: 133/999 | Loss: 0.0020 | Episodes: 17283 | Win count: 54 | Win rate: 0.719 |
time: 1.20 hours
Epoch: 134/999 | Loss: 0.0022 | Episodes: 17319 | Win count: 55 | Win rate: 0.719 |
time: 1.21 hours
Epoch: 135/999 | Loss: 0.0017 | Episodes: 17390 | Win count: 56 | Win rate: 0.719 |
time: 1.21 hours
Epoch: 136/999 | Loss: 0.0011 | Episodes: 17532 | Win count: 56 | Win rate: 0.688 |
time: 1.22 hours
Epoch: 137/999 | Loss: 0.0016 | Episodes: 17612 | Win count: 57 | Win rate: 0.719 |
time: 1.23 hours
Epoch: 138/999 | Loss: 0.0019 | Episodes: 17668 | Win count: 58 | Win rate: 0.719 |
time: 1.23 hours
Epoch: 139/999 | Loss: 0.0011 | Episodes: 17721 | Win count: 59 | Win rate: 0.719 |
time: 1.24 hours

Note: This cell will check to see if the model passes the completion check. Note: This could take several minutes.

```
In [ ]: completion_check(model, qmaze)
        show(qmaze)
```

This cell will test your model for one game. It will start the pirate at the top-left corner and run **play_game()**. The agent should find a path from the starting position to the target (treasure). The treasure is located in the bottom-right corner.

```
In [ ]: pirate_start = (0, 0)
        play_game(model, qmaze, pirate_start)
        show(qmaze)
```

Save and Submit Your Work

Hint: To use the markdown block below, double click in the **Type Markdown and LaTeX: α 2** block below, to turn it back to html, Run the cell.

After you have finished creating the code for your notebook, save your work. Make sure that your notebook contains your name in the filename (e.g. Doe_Jane_ProjectTwo.html). Download this file as an .html file clicking on **file** in *Jupyter Notebook*, navigating down to **Download as** and clicking on **.html**. Download a copy of your .html file and submit it to Brightspace.

```
In [ ]:
```